

Chuyên đề: Quy Hoạch Động Trên Tập Con (Sum Over Subsets DP - SOS DP)

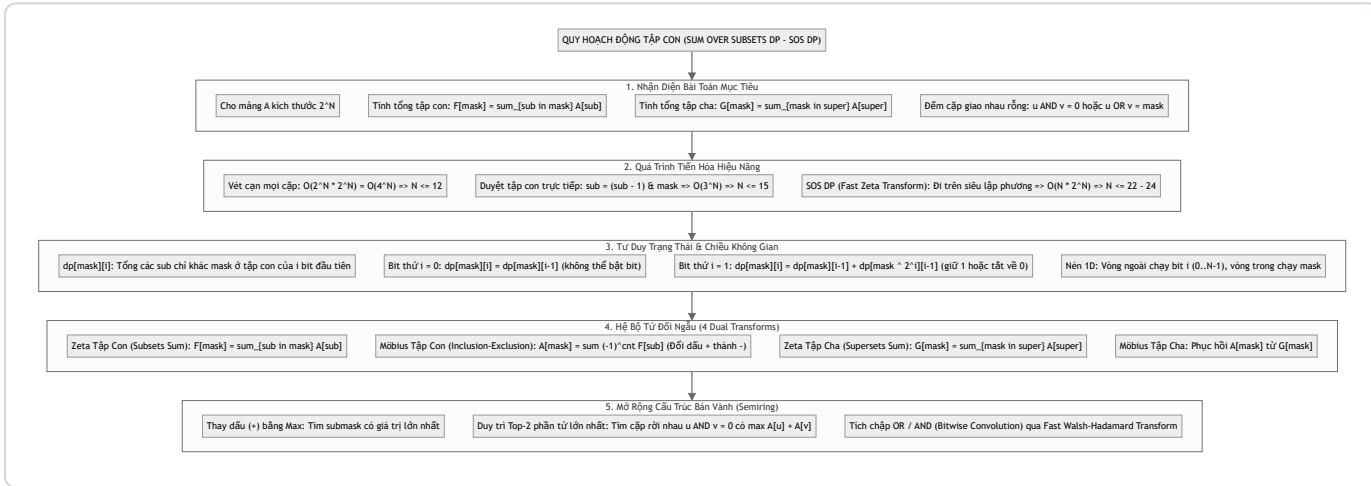
(Sum Over Subsets, Fast Zeta Transform, Fast Möbius Transform, Submask Enumeration, Hypercube DP)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Đặt Vấn Đề: Bài Toán Tính Tổng Mọi Tập Con

1.1. Mô hình bài toán gốc

Cho một mảng A gồm 2^N phần tử, được đánh chỉ số từ 0 đến $2^N - 1$. Mỗi chỉ số tương ứng với một mặt nạ nhị (bitmask) đại diện cho một tập con của tập vũ trụ $\{0, 1, \dots, N - 1\}$.

Ta cần tính toán mảng kết quả F kích thước 2^N , trong đó với mỗi mặt nạ $mask$:

$$F[mask] = \sum_{sub \subseteq mask} A[sub]$$

(Tức là $F[mask]$ là tổng giá trị $A[sub]$ của tất cả các mặt nạ sub là tập con của $mask$: $sub \& mask = sub$).

1.2. Sự bế tắc của các tiếp cận truyền thống

Cách 1: Vét cạn hai vòng lặp lồng nhau

```
for (int mask = 0; mask < (1 << n); ++mask) {
    for (int sub = 0; sub < (1 << n); ++sub) {
        if ((sub & mask) == sub) {
            F[mask] += A[sub];
        }
    }
}
```

- **Độ phức tạp:** $O(2^N \times 2^N) = O(4^N)$.
- Với $N = 20$, $4^{20} \approx 10^{12} \implies$ **TLE không thể bàn cãi**. Chỉ chạy được với $N \leq 12$.

Cách 2: Duyệt tập con tối ưu bằng Bit Trick

Duyệt trực tiếp các tập con của $mask$ bằng công thức `sub = (sub - 1) & mask`:

```
for (int mask = 0; mask < (1 << n); ++mask) {
    F[mask] = A[0];
    for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
        F[mask] += A[sub];
    }
}
```

- **Độ phức tạp:** Số lần lặp chính xác là $\sum_{k=0}^N \binom{N}{k} 2^k = (1 + 2)^N = 3^N$.
- Với $N = 20$, $3^{20} \approx 3.48 \times 10^9 \implies$ **vẫn TLE** trong giới hạn thời gian 1.0s - 2.0s của kỳ thi lập trình. Cách này chỉ áp dụng được với $N \leq 15$.

Cách 3: Sum Over Subsets DP (SOS DP / Fast Zeta Transform)

- **Độ phức tạp:** $O(N \cdot 2^N)$.
- Với $N = 20$: $20 \times 2^{20} \approx 20 \times 10^6 \approx 2 \times 10^7$ phép tính $\implies$ **Chạy mượt mà trong chưa đầy 0.05 giây!** Hỗ trợ tốt đến $N = 22 \dots 24$.

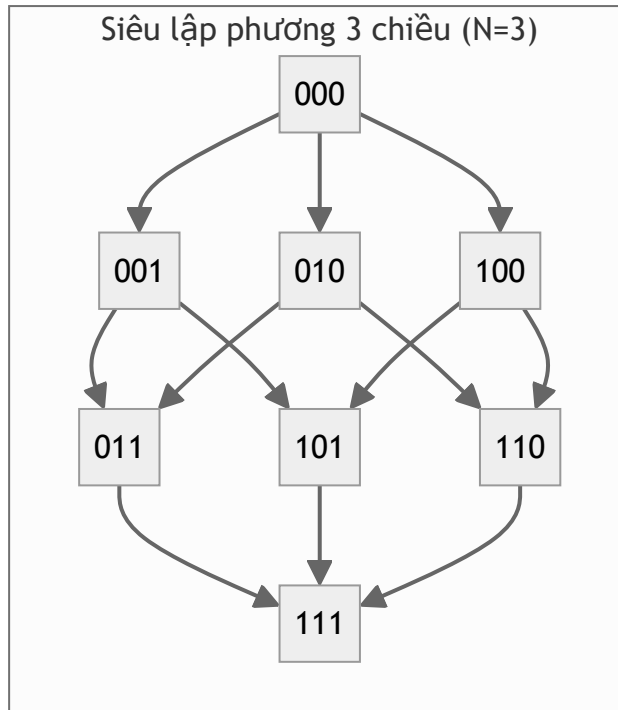
2. Bản Chất Toán Học: Đi Trên Siêu Lập Phương N Chiều

2.1. Không gian Siêu lập phương (Boolean Hypercube $\{0, 1\}^N$)

Mỗi mặt nạ bit có độ dài N là một đỉnh của một **siêu lập phương N chiều**.

Hai mặt nạ kề nhau trên siêu lập phương khi và chỉ khi chúng khác nhau đúng **1 bit** (khoảng cách Hamming bằng 1).

Thay vì nhảy vọt từ một mặt nạ đến tất cả các tập con của nó trong một bước (như cách $O(3^N)$), SOS DP thực hiện chuyển trạng thái **từng chiều một** (từ chiều 0 đến chiều $N - 1$).



2.2. Định nghĩa Hàm Trạng Thái 2 Chiều

Gọi $dp[mask][i]$ là tổng của tất cả các $A[sub]$ sao cho:

1. $sub \subseteq mask$.
2. sub và $mask$ **chỉ có thể khác nhau ở $i + 1$ bit đầu tiên** (các bit có chỉ số từ 0 đến i). Đối với các bit từ $i + 1$ đến $N - 1$, sub và $mask$ bắt buộc phải giống hệt nhau!

Trạng thái cơ sở (Base Case):

Khi $i = -1$ (không bit nào được phép khác nhau):

$$dp[mask][-1] = A[mask]$$

Công thức chuyển trạng thái (State Transition):

Xét bit thứ i của $mask$:

- **Trường hợp 1: Bit thứ i của $mask$ là 0** ($!(mask \& (1 \ll i))$):

Vì $sub \subseteq mask$, bit thứ i của sub bắt buộc phải bằng 0. Do đó, ta không có lựa chọn nào khác ngoài việc giữ nguyên bit thứ i là 0.

$$dp[mask][i] = dp[mask][i - 1]$$

- **Trường hợp 2: Bit thứ i của $mask$ là 1** ($mask \& (1 \ll i)$):

Vì bit thứ i của $mask$ là 1, bit thứ i của sub có thể bằng 1 **hoặc** bằng 0:

- Nếu bit thứ i của sub là 1: Tập các sub này chính là các tập con được tính trong $dp[mask][i - 1]$.

- Nếu bit thứ i của sub là 0: Tập các sub này tương ứng với việc tắt bit thứ i của $mask$ về 0, tức là $dp[mask \oplus 2^i][i - 1]$.

Do hai tập hợp này hoàn toàn rời nhau, ta có:

$$dp[mask][i] = dp[mask][i - 1] + dp[mask \oplus 2^i][i - 1]$$

Kết quả cuối cùng:

Sau khi xét hết cả N chiều (từ $i = 0$ đến $N - 1$):

$$F[mask] = dp[mask][N - 1]$$

3. Tối Ưu Hóa Bộ Nhớ: Nén Không Gian Về Mảng 1D Tuyệt Đối

Nhìn vào công thức chuyển trạng thái:

$$dp[mask][i] = dp[mask][i - 1] + (bit_i == 1 ? dp[mask \oplus 2^i][i - 1] : 0)$$

Ta nhận thấy tầng i chỉ phụ thuộc duy nhất vào tầng $i - 1$.

Đặc biệt, $mask \oplus 2^i$ luôn có giá trị **nhỏ hơn** $mask$ (vì bit thứ i của nó bằng 0).

Nếu ta đảo ngược vòng lặp: **Cho vòng lặp bit i ra ngoài cùng**, và vòng lặp $mask$ vào bên trong, ta có thể ghi đè trực tiếp trên **đúng 1 mảng 1 chiều** $dp[mask]$!

```
// Khởi tạo dp với mảng gốc A
for (int mask = 0; mask < (1 << n); ++mask) {
    dp[mask] = A[mask];
}

// SOS DP Engine: O(N * 2^N) thời gian, O(2^N) bộ nhớ
for (int i = 0; i < n; ++i) {
    for (int mask = 0; mask < (1 << n); ++mask) {
        if (mask & (1 << i)) {
            dp[mask] += dp[mask ^ (1 << i)];
        }
    }
}
```

🚩 Quan trọng:

****Quy tắc Bất di bất dịch về Thứ tự Vòng lặp:****

- ****Vòng lặp bit i BẮT BUỘC phải nằm ở ngoài cùng!****

- Nếu đưa vòng lặp $mask$ ra ngoài và lặp bit i bên trong, bạn sẽ tính lặp các tập con nhiều lần và cho ra kết quả hoàn toàn sai lệch!

- Việc chạy bit i ở ngoài cùng tương đương với việc thực hiện Biến đổi Zeta nhanh (Fast Zeta Transform) trên từng chiều của siêu lập phương.

4. Hệ Bộ Tứ Đối Ngẫu (The 4 Dual Transforms of SOS DP)

Trong giải thuật thi đấu, ta không chỉ gặp bài toán tính tổng tập con mà còn gặp bài toán tính tổng tập cha và bài toán khôi phục mảng ban đầu (Nghịch đảo Möbius). Bảng dưới đây tổng hợp đầy đủ 4 biến thể:

4.1. Bảng Tổng Hợp 4 Dạng Biến Đổi Siêu Lập Phương

Tên gọi toán học	Bài toán cần tính	Điều kiện if	Phép toán cập nhật	Ý nghĩa thực chiến
1. Zeta Tập con (SOS DP Subsets)	$F[mask] = \sum_{sub \subseteq mask} A[sub]$	<code>mask & (1 << i)</code>	<code>dp[mask] += dp[mask ^ (1 << i)]</code>	Tính tổng mọi tập con
2. Möbius Tập con (Inverse Subsets)	$A[mask] = \sum_{sub \subseteq mask} (-1)^{ mask \setminus sub } F[sub]$	<code>mask & (1 << i)</code>	<code>dp[mask] -= dp[mask ^ (1 << i)]</code>	Phục hồi mảng A từ F (Bảo hàm - Loại trừ)
3. Zeta Tập cha (SOS DP Supersets)	$G[mask] = \sum_{mask \subseteq super} A[super]$	<code>!(mask & (1 << i))</code>	<code>dp[mask] += dp[mask ^ (1 << i)]</code>	Tính tổng mọi tập cha
4. Möbius Tập cha (Inverse Supersets)	$A[mask] = \sum_{mask \subseteq super} (-1)^{ super \setminus mask } G[super]$	<code>!(mask & (1 << i))</code>	<code>dp[mask] -= dp[mask ^ (1 << i)]</code>	Phục hồi mảng A từ G

4.2. Khung mã nguồn trực quan cho cả 4 biến thể

```
namespace cp::sos {

// 1. Tổng trên tập con:  $F[mask] = \sum_{sub \subseteq mask} A[sub]$ 
void sos_subsets_sum(std::vector<long long>& dp, int n) {
    for (int i = 0; i < n; ++i)
        for (int mask = 0; mask < (1 << n); ++mask)
            if (mask & (1 << i))
                dp[mask] += dp[mask ^ (1 << i)];
}

// 2. Nghịch đảo tập con: Thu hồi  $A[mask]$  từ  $F[mask]$  (Inclusion-Exclusion)
void sos_subsets_inverse(std::vector<long long>& dp, int n) {
    for (int i = 0; i < n; ++i)
        for (int mask = 0; mask < (1 << n); ++mask)
            if (mask & (1 << i))
                dp[mask] -= dp[mask ^ (1 << i)]; // Chỉ đổi dấu + thành - !
}

// 3. Tổng trên tập cha:  $G[mask] = \sum_{mask \subseteq super} A[super]$ 
void sos_supersets_sum(std::vector<long long>& dp, int n) {
    for (int i = 0; i < n; ++i)
        for (int mask = (1 << n) - 1; mask >= 0; --mask)
            if (!(mask & (1 << i)))
                dp[mask] += dp[mask ^ (1 << i)];
}

// 4. Nghịch đảo tập cha: Thu hồi  $A[mask]$  từ  $G[mask]$ 
void sos_supersets_inverse(std::vector<long long>& dp, int n) {
    for (int i = 0; i < n; ++i)
        for (int mask = (1 << n) - 1; mask >= 0; --mask)
            if (!(mask & (1 << i)))
                dp[mask] -= dp[mask ^ (1 << i)];
}

} // namespace cp::sos
```

5. Mở Rộng Bán Vành (Semiring Extensions) & Bài Toán Hai Phần Tử Rời Nhau

SOS DP không bị giới hạn ở phép cộng thông thường! Vì tính chất của các tập con được phân rã theo chiều, ta có thể thay thế phép cộng bằng bất kỳ phép toán nào thỏa mãn tính chất kết hợp (Semiring), hoặc duy trì một **cấu trúc dữ liệu thu nhỏ** trong mỗi ô trạng thái.

5.1. Thay phép cộng bằng phép lấy cực đại (SOS DP Max)

Nếu cần tìm $F[mask] = \max_{sub \subseteq mask} A[sub]$:

```

for (int i = 0; i < n; ++i) {
    for (int mask = 0; mask < (1 << n); ++mask) {
        if (mask & (1 << i)) {
            dp[mask] = std::max(dp[mask], dp[mask ^ (1 << i)]);
        }
    }
}

```

5.2. Bài toán kinh điển: Tìm cặp rời nhau có tổng lớn nhất

Đề bài: Cho mảng A gồm M số nguyên. Tìm 2 số x, y sao cho $x \& y = 0$ và $x + y$ (hoặc $A[x] + A[y]$) đạt giá trị lớn nhất.

Phân tích:

- Điều kiện $x \& y = 0 \iff y \subseteq (\sim x) \iff y \subseteq ((2^N - 1) \oplus x)$.
- Với mỗi số x , ta cần tìm giá trị $A[y]$ lớn nhất sao cho y là **tập con của phần bù của x** .
- Đặt $inv_x = ((1 \ll N) - 1) \oplus x$. Bài toán quy về:

$$\max_x \left(A[x] + \max_{y \subseteq inv_x} A[y] \right)$$

- Sử dụng SOS DP Max để tiền xử lý $\max_{y \subseteq mask} A[y]$ cho mọi $mask$ trong $O(N \cdot 2^N)$. Sau đó duyệt qua từng số x trong $O(1)$!

Khi hai phần tử có thể trùng vị trí (Duy trì Top-2 lớn nhất):

Nếu mỗi ô mảng chỉ được dùng 1 lần, ta có thể duy trì **cặp 2 phần tử lớn nhất từ các chỉ số khác nhau**

`pair<int, int> best[mask] :`

```

struct Top2 {
    int max1 = -1, id1 = -1;
    int max2 = -1, id2 = -1;

    void insert(int val, int id) {
        if (val > max1) {
            if (id != id1) { max2 = max1; id2 = id1; }
            max1 = val; id1 = id;
        } else if (val > max2 && id != id1) {
            max2 = val; id2 = id;
        }
    }

    void merge(const Top2& other) {
        if (other.id1 != -1) insert(other.max1, other.id1);
        if (other.id2 != -1) insert(other.max2, other.id2);
    }
};

```

Trong SOS DP, ta gộp:

```
if (mask & (1 << i)) {
    best[mask].merge(best[mask ^ (1 << i)]);
}
```

6. Mối Quan Hệ Giữa SOS DP Và Tích Chập Bitwise (FWHT)

6.1. Tích chập OR (OR Convolution)

Cho 2 mảng A và B . Cần tính mảng C sao cho:

$$C[k] = \sum_{i|j=k} A[i] \cdot B[j]$$

Ta có tính chất đẹp để sau:

$$i \subseteq k \text{ và } j \subseteq k \iff (i | j) \subseteq k$$

Do đó:

$$\sum_{sub \subseteq k} C[sub] = \left(\sum_{sub \subseteq k} A[sub] \right) \cdot \left(\sum_{sub \subseteq k} B[sub] \right)$$

Quy trình giải bằng SOS DP trong 3 bước:

1. Áp dụng **Zeta Tập con (SOS DP Subsets Sum)** lên mảng A và mảng B để nhận được $\hat{A}$ và $\hat{B}$ trong $O(N \cdot 2^N)$.
2. Nhân từng phần tử: $\hat{C}[mask] = \hat{A}[mask] \cdot \hat{B}[mask]$ trong $O(2^N)$.
3. Áp dụng **Möbius Tập con (SOS DP Subsets Inverse)** lên mảng $\hat{C}$ để khôi phục lại mảng kết quả C trong $O(N \cdot 2^N)$!

Kết luận: SOS DP và Nghịch đảo của nó chính là bản chất của **Biến đổi Walsh-Hadamard nhanh (FWHT)** đối với phép toán OR và AND!

7. Cạm Bẫy Phòng Thi & Kỹ Thuật Tối Ưu Tột Đỉnh

7.1. Cạm bẫy tràn số (Integer Overflow)

Tổng trên tập con $F[mask]$ tăng theo cấp số nhân. Dù các phần tử ban đầu $A[i] \leq 10^9$, khi cộng dồn qua 2^N phần tử, kết quả có thể vượt quá $2^{63} - 1$.

- Luôn sử dụng `long long` hoặc làm việc trên vành số học Modulo ($10^9 + 7$ hoặc 998244353).

- Khi dùng Modulo, trong hàm nghịch đảo `sos_subsets_inverse`, nhớ xử lý số âm:

```
dp[mask] = (dp[mask] - dp[mask ^ (1 << i)] + MOD) % MOD;
```

7.2. Tối ưu Cache CPU với thứ tự lặp mảng

Trong vòng lặp:

```
for (int i = 0; i < n; ++i) {
    for (int mask = 0; mask < (1 << n); ++mask) {
        if (mask & (1 << i)) {
            dp[mask] += dp[mask ^ (1 << i)];
        }
    }
}
```

Mảng được duyệt tuần tự từ 0 đến $2^N - 1$. Trình biên dịch có thể tự động vector hóa (Auto-vectorization) với tập lệnh SIMD (AVX2).

Tốc độ thực thi cho $N = 22$ (4×10^6 phần tử) chỉ mất khoảng **0.08 giây**!

8. Bảng So Sánh Các Kỹ Thuật Xử Lý Mặt Nạ Bit

Kỹ thuật	Bản chất toán học	Độ phức tạp Thời gian	Độ phức tạp Bộ nhớ	Phạm vi N tối đa
Duyệt trâu mọi cặp	Lặp lồng nhau 2 lần	$O(4^N)$	$O(2^N)$	$N \leq 12$
Submask Iteration	<code>sub = (sub - 1) & mask</code>	$O(3^N)$	$O(2^N)$	$N \leq 15$
SOS DP / Fast Zeta	Phân rã theo chiều trên Siêu lập phương	$O(N \cdot 2^N)$	$O(2^N)$ phẳng	$N \leq 22 \dots 24$
FWHT (Walsh-Hadamard)	Biến đổi Fourier trên nhóm $(\mathbb{Z}_2)^N$	$O(N \cdot 2^N)$	$O(2^N)$	$N \leq 22 \dots 24$

9. Tài liệu Tham khảo & Đọc thêm

1. **Codeforces Community:**
2. [SOS Dynamic Programming Tutorial](#) — Hướng dẫn kinh điển về Fast Zeta Transform trên siêu lập phương.
3. **VNOI Wiki:**
4. [Quy hoạch động SOS \(Phần 1\)](#) — Bản chất bài toán tổng trên tập con $O(N \cdot 2^N)$.
5. [Quy hoạch động SOS \(Phần 2\)](#) — Mở rộng tích chập bitwise FWHT.
6. **CP-Algorithms:**
7. [Submask Enumeration](#) — Duyệt mọi tập con của một mặt nạ trong $O(3^N)$.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.